

DevOps Capstone Project:

Deployment of Online Ticketing Book-My-Show

Submitted to - Vikul Sir

Date of Submission – 26/MAY/2026

Submitted by – Rahul Nalathathi

Project Overview:

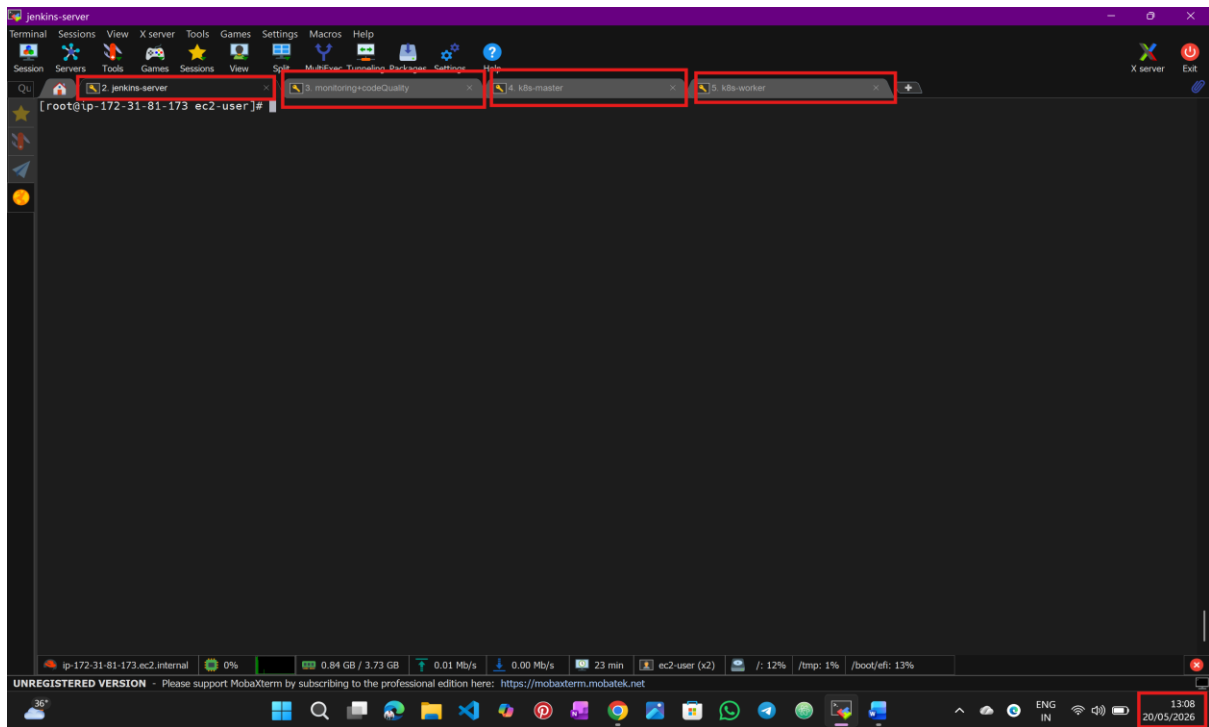
Book-My-Show is a popular online movie ticketing platform that allows users to browse movies, book tickets, and manage their bookings. The company is transitioning from a monolithic architecture to a microservices-based architecture to improve scalability, deployment efficiency, and infrastructure management.

Step: 1 Login into AWS console and launch four (4) EC2 instances

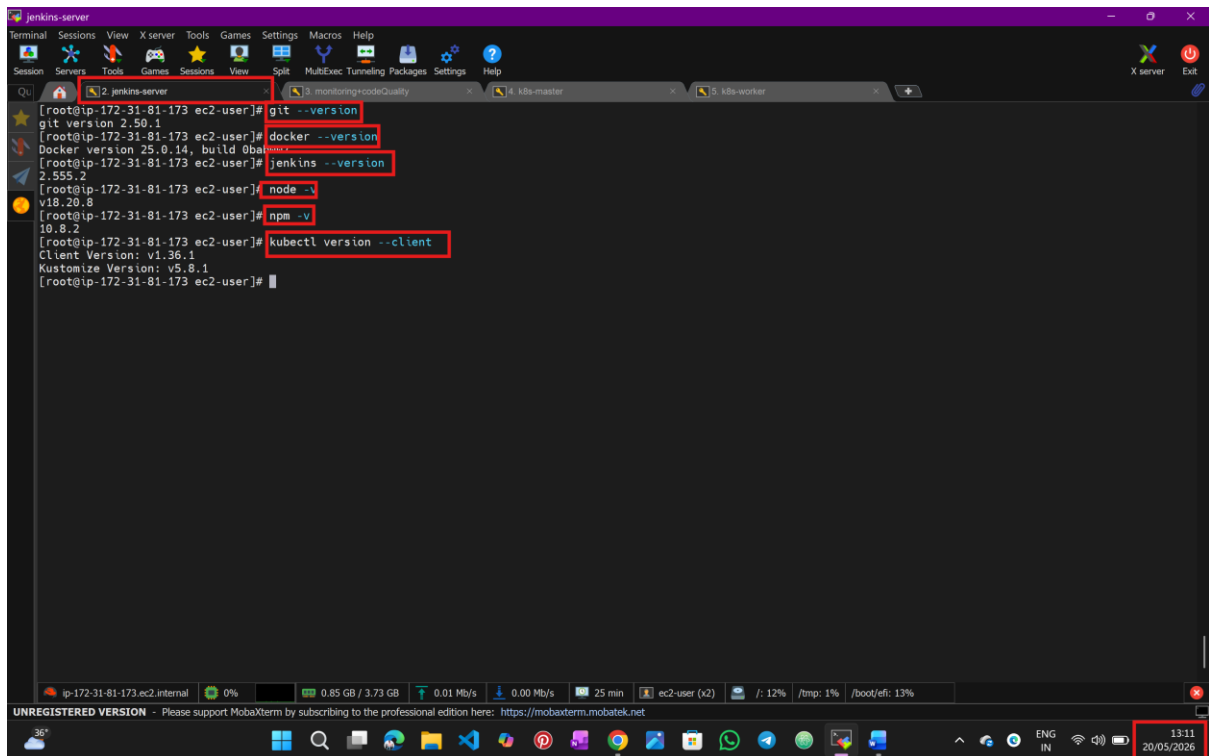
The screenshot displays the AWS Management Console interface for the 'us-east-1' region. The 'Instances' page shows four EC2 instances in a 'Running' state. The instances are named 'jenkins-server', 'monitoring+code quality', 'k8s-Master', and 'k8s-worker'. Each instance is of type 'c7i-flex.large' and is located in the 'us-east-1b' availability zone. The console also shows monitoring graphs for CPU utilization, network in/out, and network packets in/out for the selected instances.

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...
jenkins-server	i-008396ed2c653299c	Running	c7i-flex.large	3/3 checks passed	us-east-1b	ec2-44-211-246-89...	44.211.246.89
monitoring+code quality	i-0583bf3511272326	Running	c7i-flex.large	3/3 checks passed	us-east-1b	ec2-52-203-99-215...	52.203.99.215
k8s-Master	i-0ea808b58e89b278d	Running	c7i-flex.large	3/3 checks passed	us-east-1b	ec2-3-95-189-115...	3.95.189.115
k8s-worker	i-010967ab978c4e8ac	Running	c7i-flex.large	3/3 checks passed	us-east-1b	ec2-44-212-61-74...	44.212.61.74

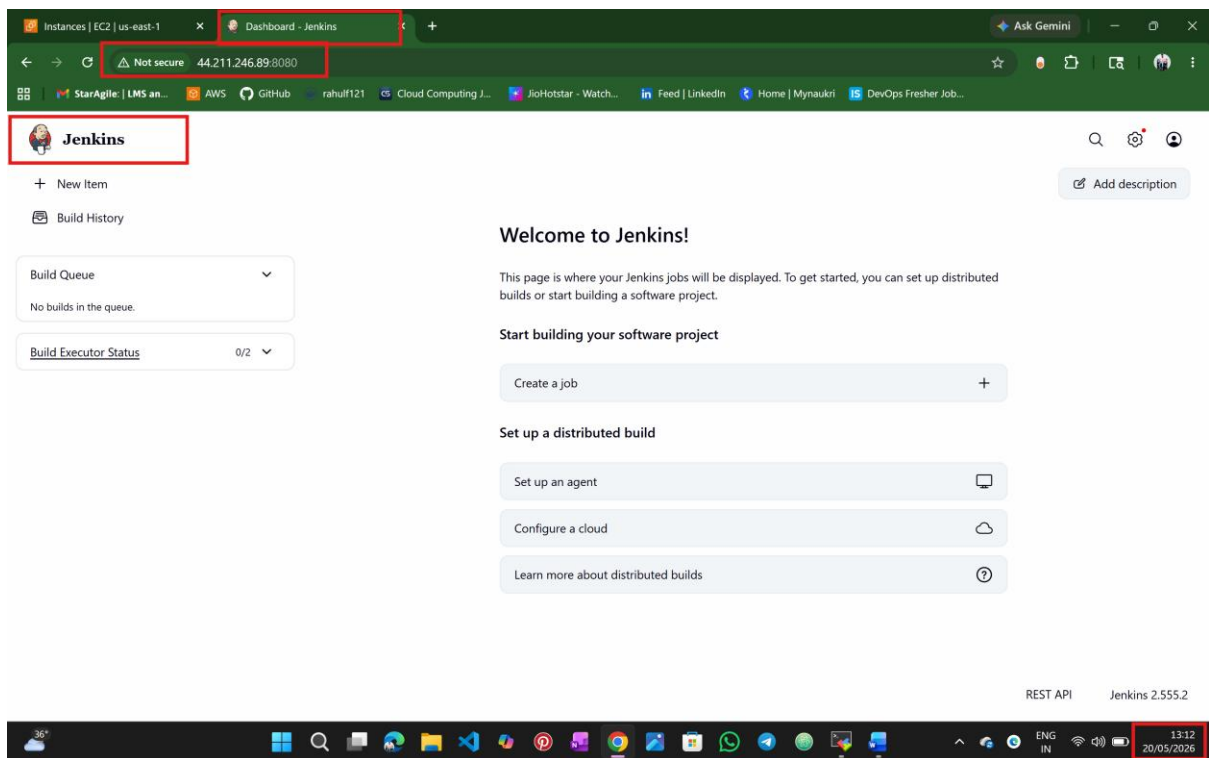
Step:2 connect with ssh via Mobaxterm



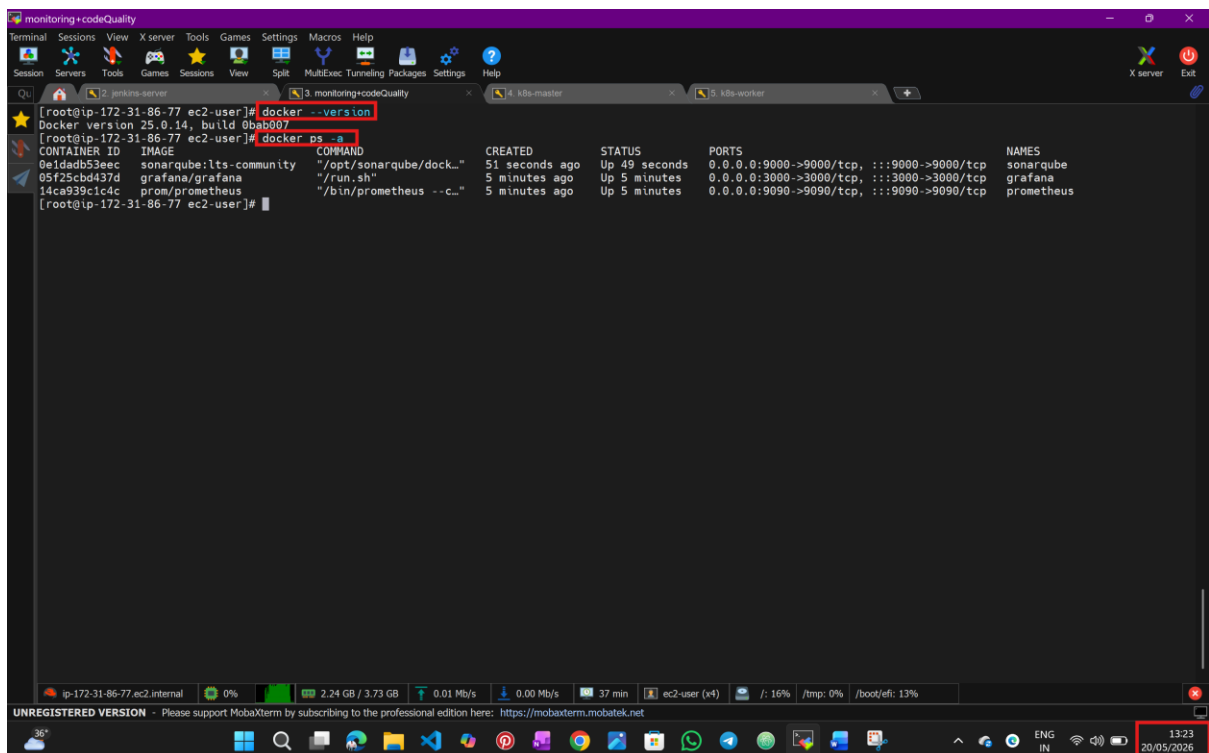
Step :3 Install prerequisite in Jenkins server



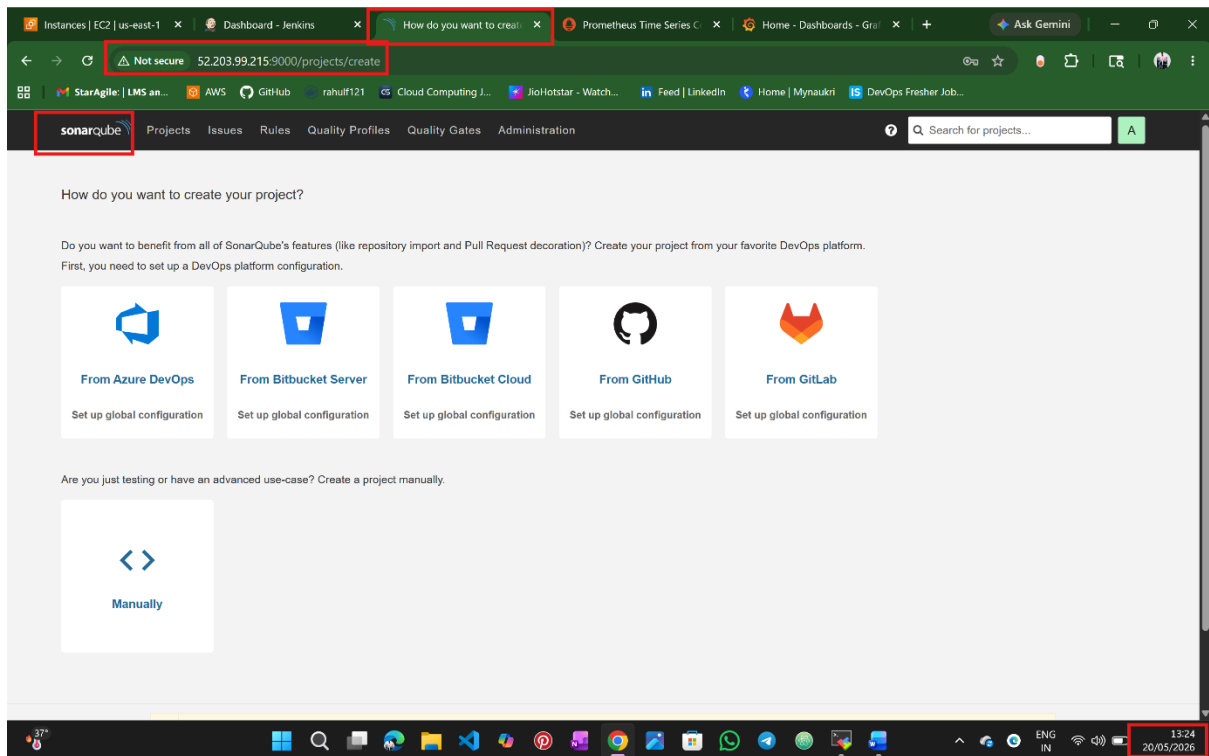
Step: access the Jenkins webpage and install available plugins



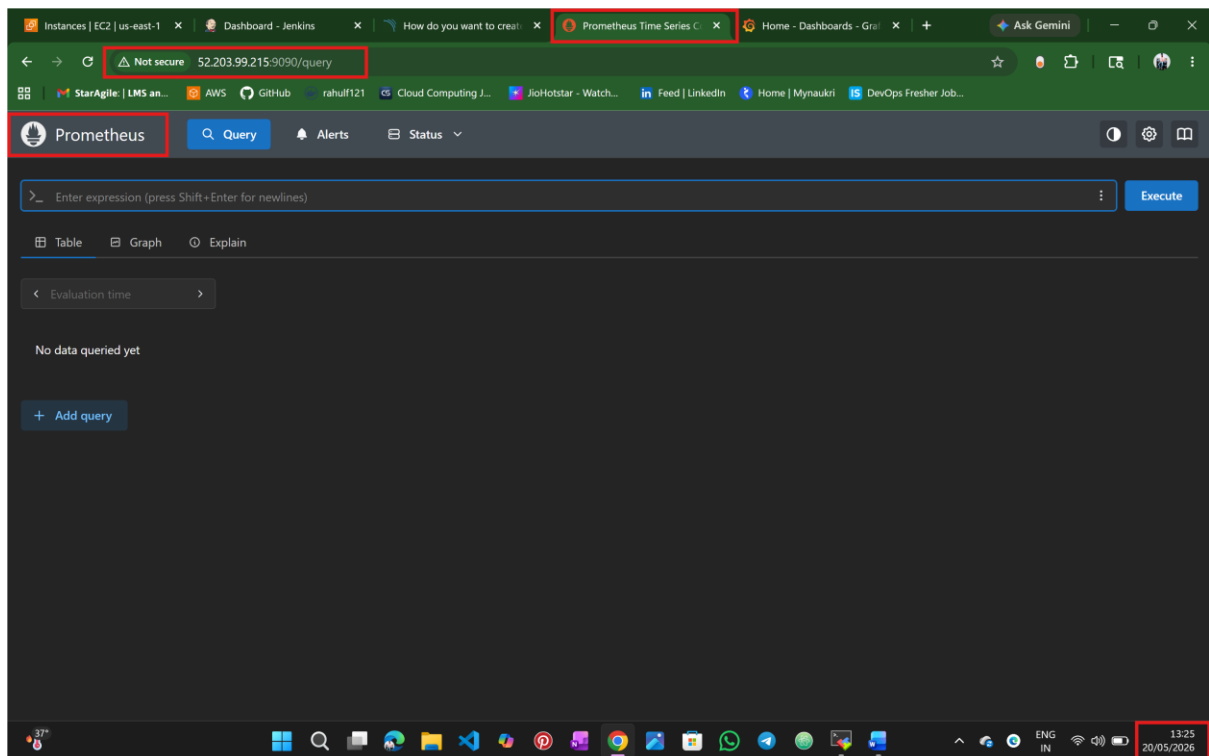
Step:3 In server 2, install Docker, SonarQube, Prometheus and Grafana



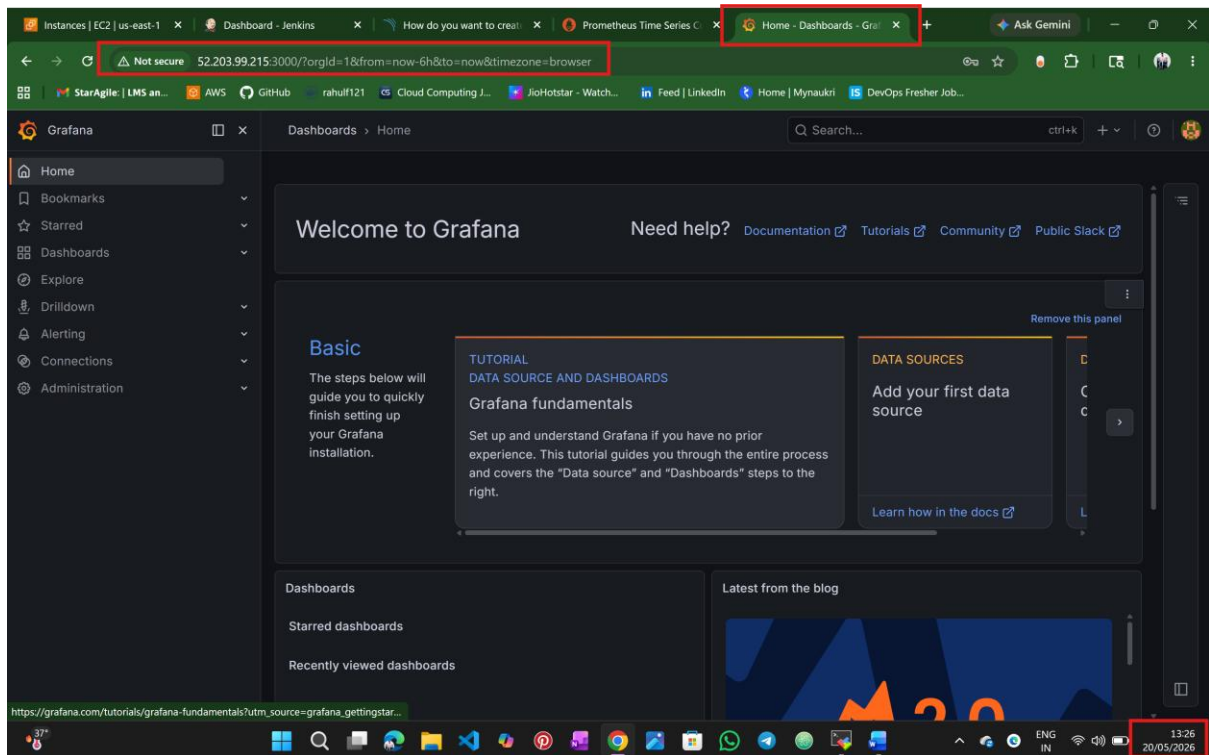
Step :4 Access the SonarQube webpage



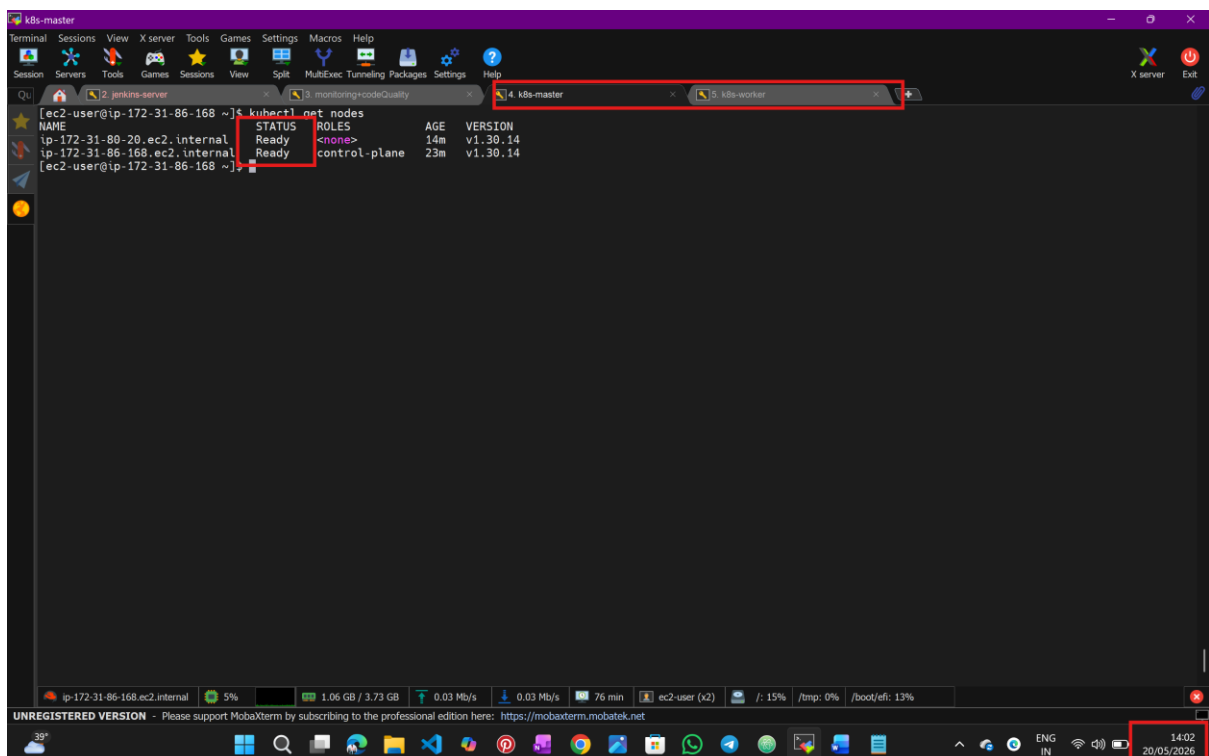
Step: 5 Access the Prometheus webpage



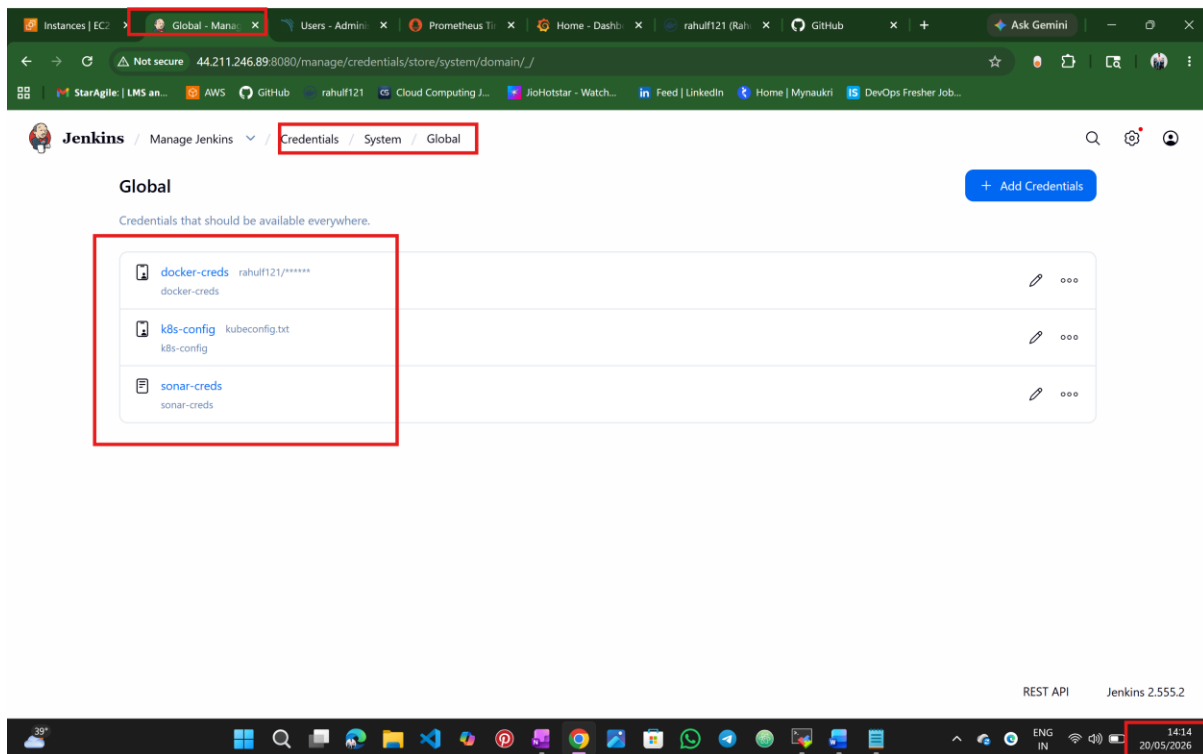
Step :6 Access Grafana webpage



Step :7 in server 3, which is k8s master node. Install prerequisite



Step:8 Store docker, SonarQube and k8s config credentials in Jenkins GUI

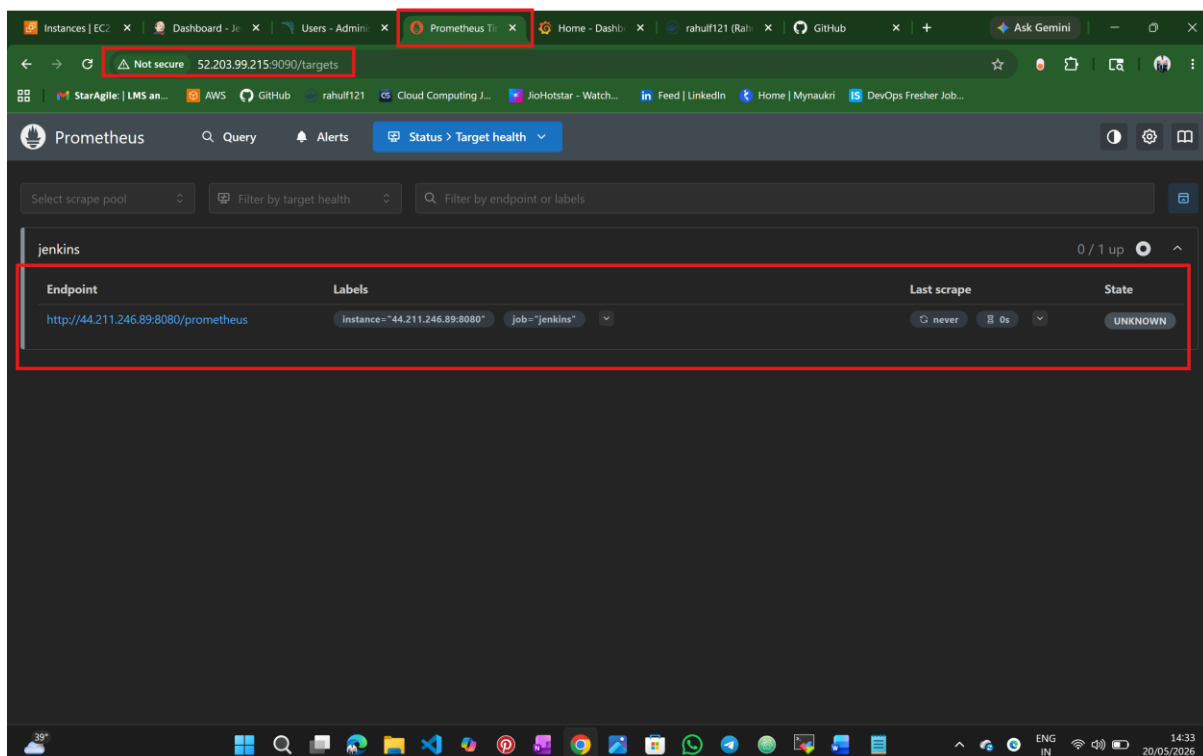


The screenshot shows the Jenkins GUI's 'Global' credentials page. The breadcrumb navigation at the top indicates the path: 'Credentials / System / Global'. A red box highlights the 'Global' tab in the breadcrumb. The main heading is 'Global', with a subtext 'Credentials that should be available everywhere.' and a '+ Add Credentials' button. A table lists three credentials, which are highlighted by a red box:

Name	Username	Kind
docker-creds	rahulf121/*****	docker-creds
k8s-config	kubeconfig.txt	k8s-config
sonar-creds	sonar-creds	sonar-creds

At the bottom right, the status 'Jenkins 2.555.2' is visible. The Windows taskbar at the bottom shows the time as 14:14 on 20/05/2026.

Step:9 connect Jenkins server and Prometheus server

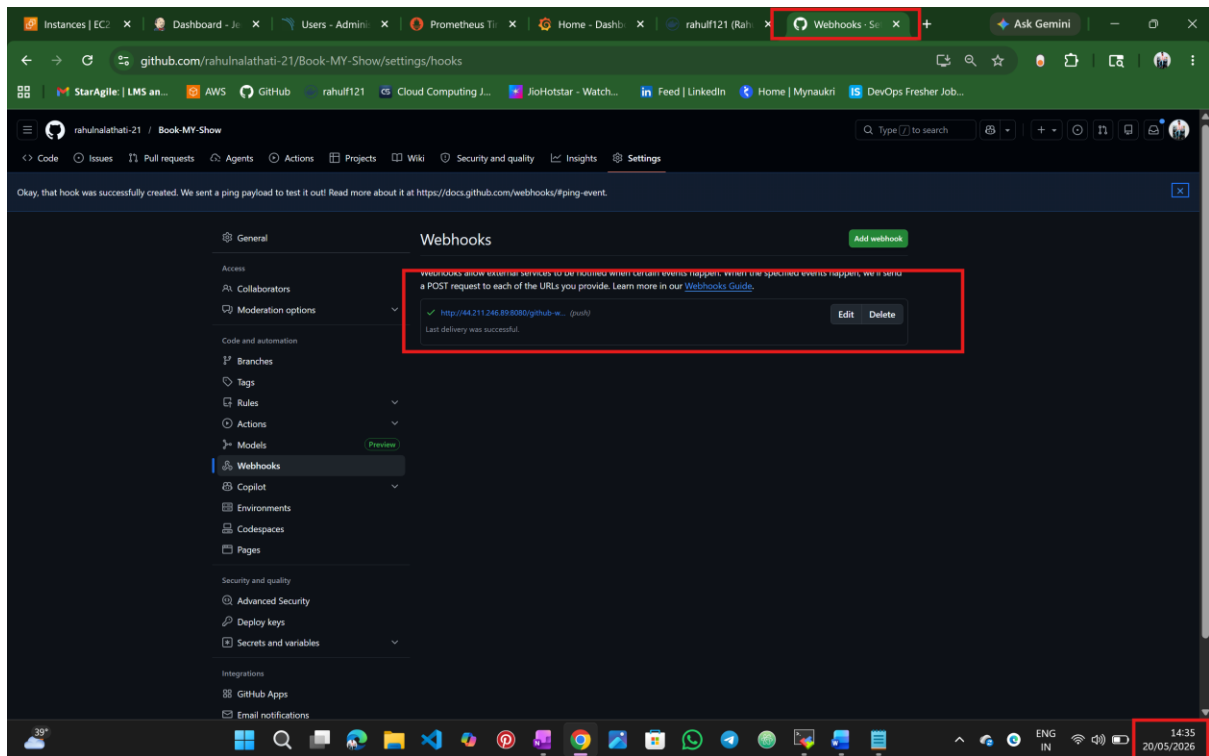


The screenshot shows the Prometheus 'Targets' page. The breadcrumb navigation at the top indicates the path: 'Prometheus / Targets'. A red box highlights the 'Prometheus' tab in the breadcrumb. The main heading is 'Prometheus', with a subtext 'Targets' and a 'Status > Target health' dropdown. The page shows a list of targets, with one target highlighted by a red box:

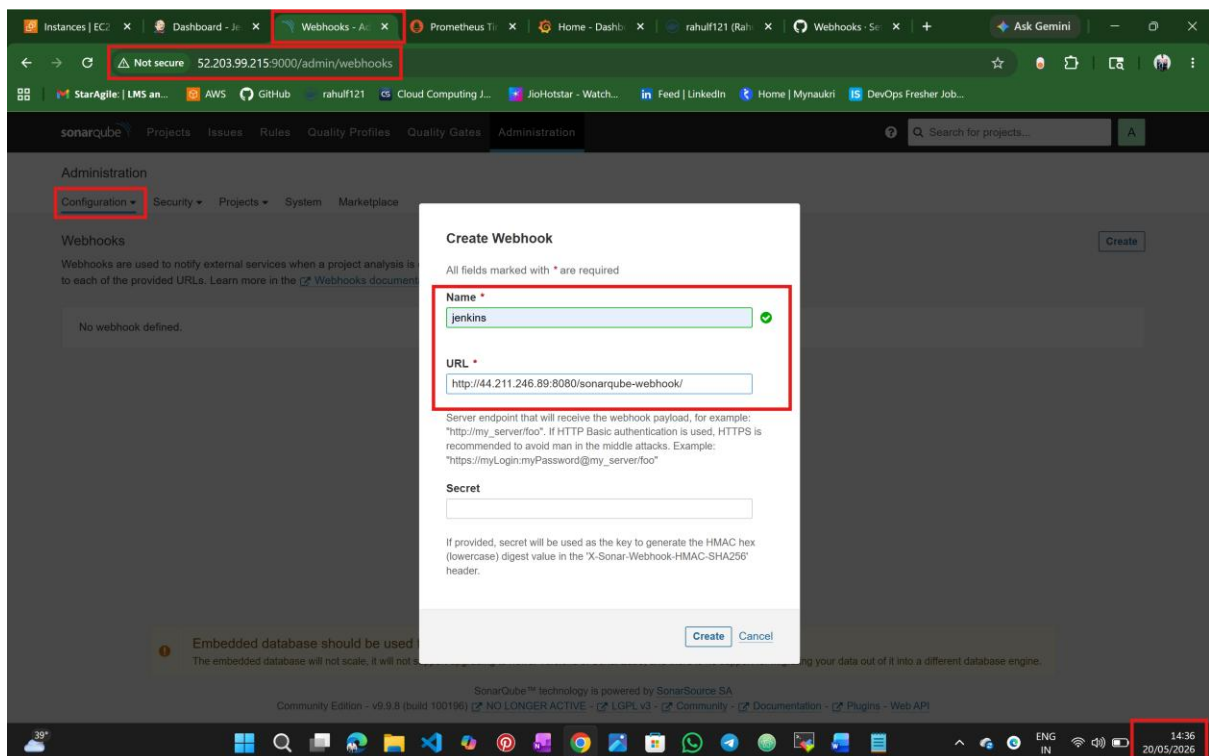
Endpoint	Labels	Last scrape	State
http://44.211.246.89:8080/prometheus	Instance="44.211.246.89:8080" Job="jenkins"	never 0s	UNKNOWN

The Windows taskbar at the bottom shows the time as 14:33 on 20/05/2026.

Step:10 connect the Jenkins URL in Git hub webhook



Step:11 connect SonarQube and Jenkins



Step: 12 Now Build the job

The screenshot shows the Jenkins web interface for a pipeline named 'book-my-show-APP'. The left sidebar contains navigation options: Status, Changes, Build Now (highlighted with a red box), Configure, Delete Pipeline, Full Stage View, Stages, Rename, Pipeline Syntax, and GitHub Hook Log. The main area displays a table of pipeline stages and their durations. Below the table is a 'Permalinks' section showing a build history with a red box around the first build (#1) at 9:11 AM. The bottom status bar shows the REST API, Jenkins version 2.555.2, and the system clock.

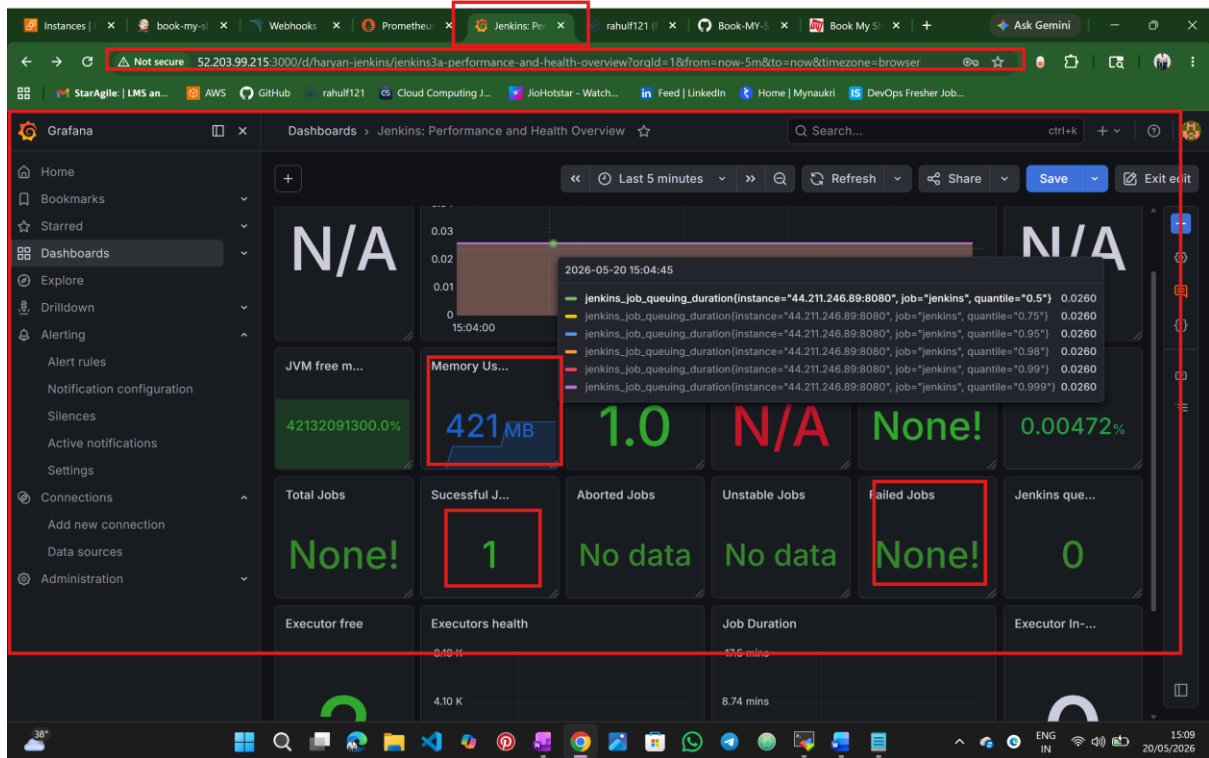
Stage	Duration
one sitory	3ms
Verify Project Structure	494ms
Install Dependencies	39s
SonarQube Analysis	36s
Build React App	50s
Build Docker Image	3min 11s
DockerHub Login	630ms
Push Docker Image	6s
Deploy to Kubernetes	1s
Verify Deployment	464ms
Declarative: Post Actions	352ms

Builds > Today
#1 9:11 AM

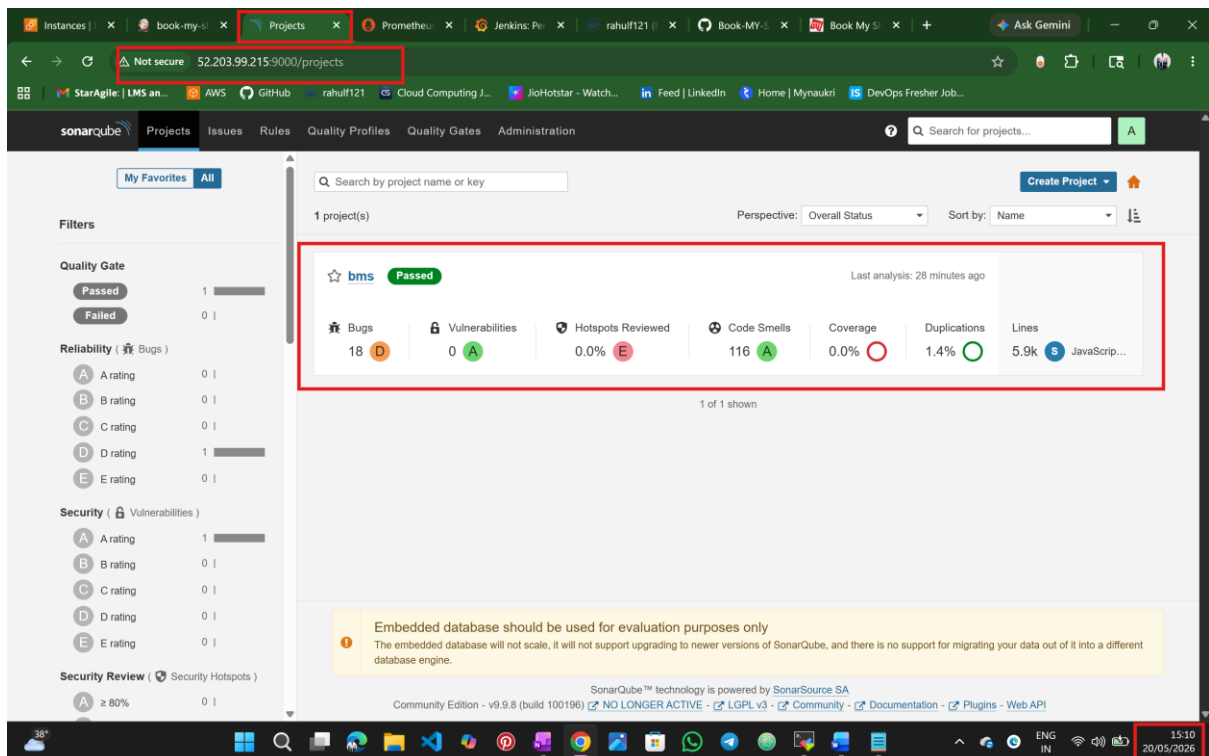
Step:13 Access the application using worker node IP address: port no

The screenshot shows the 'book my show' application running on a worker node. The browser address bar displays the IP address 44.212.61.74:30007. The application interface includes a search bar, navigation links (Movies, Stream, Events, Plays, Sports, Activities, Fanhood, Buzz), and a 'Sign In' button. Below the navigation bar, there are promotional banners for 'ENJOY A KILLER COLLECTION OF MOVIES' and 'WATCH ON book my show ONLINE'. The main content area features 'Recommended Movies' and 'The Best of Entertainment' sections. The bottom status bar shows the system clock at 15:05 on 20/05/2026.

Step:14 Now access the Grafana visualization Dashboard



Step: 15 This is the Code Quality analysis



Step:16 This is the Prometheus metrics

```
# HELP jvm_memory_objects_pending_finalization The number of objects waiting in the finalizer queue.
# TYPE jvm_memory_objects_pending_finalization gauge
jvm_memory_objects_pending_finalization{area="heap"} 0.0
# HELP jvm_memory_bytes_used Used bytes of a given JVM memory area.
# TYPE jvm_memory_bytes_used gauge
jvm_memory_bytes_used{area="heap"} 1.78583264E8
jvm_memory_bytes_used{area="nonheap"} 1.86708072E8
# HELP jvm_memory_bytes_committed Committed (bytes) of a given JVM memory area.
# TYPE jvm_memory_bytes_committed gauge
jvm_memory_bytes_committed{area="heap"} 2.8311552E8
jvm_memory_bytes_committed{area="nonheap"} 1.95690496E8
# HELP jvm_memory_bytes_max Max (bytes) of a given JVM memory area.
# TYPE jvm_memory_bytes_max gauge
jvm_memory_bytes_max{area="heap"} 1.002438656E9
jvm_memory_bytes_max{area="nonheap"} -1.0
# HELP jvm_memory_bytes_init Initial bytes of a given JVM memory area.
# TYPE jvm_memory_bytes_init gauge
jvm_memory_bytes_init{area="heap"} 6.291456E7
jvm_memory_bytes_init{area="nonheap"} 7667712.0
# HELP jvm_memory_pool_bytes_used Used bytes of a given JVM memory pool.
# TYPE jvm_memory_pool_bytes_used gauge
jvm_memory_pool_bytes_used{pool="CodeHeap 'non-nmethods'"} 3402624.0
jvm_memory_pool_bytes_used{pool="Metaspace"} 1.29202456E8
jvm_memory_pool_bytes_used{pool="CodeHeap 'profiled nmethods'"} 2.5494144E7
jvm_memory_pool_bytes_used{pool="Compressed Class Space"} 1.593416E7
jvm_memory_pool_bytes_used{pool="G1 Eden Space"} 1.7825792E7
jvm_memory_pool_bytes_used{pool="G1 Old Gen"} 1.54608192E9
jvm_memory_pool_bytes_used{pool="G1 Survivor Space"} 6157280.0
jvm_memory_pool_bytes_used{pool="CodeHeap 'non-profiled nmethods'"} 1.2674688E7
# HELP jvm_memory_pool_bytes_committed Committed bytes of a given JVM memory pool.
# TYPE jvm_memory_pool_bytes_committed gauge
jvm_memory_pool_bytes_committed{pool="CodeHeap 'non-nmethods'"} 3604480.0
jvm_memory_pool_bytes_committed{pool="Metaspace"} 1.32972544E8
jvm_memory_pool_bytes_committed{pool="CodeHeap 'profiled nmethods'"} 2.8639232E7
jvm_memory_pool_bytes_committed{pool="Compressed Class Space"} 1.7760256E7
jvm_memory_pool_bytes_committed{pool="G1 Eden Space"} 1.13246208E8
jvm_memory_pool_bytes_committed{pool="G1 Old Gen"} 1.63577856E9
jvm_memory_pool_bytes_committed{pool="G1 Survivor Space"} 6291456.0
jvm_memory_pool_bytes_committed{pool="CodeHeap 'non-profiled nmethods'"} 1.2713984E7
# HELP jvm_memory_pool_bytes_max Max bytes of a given JVM memory pool.
# TYPE jvm_memory_pool_bytes_max gauge
jvm_memory_pool_bytes_max{pool="CodeHeap 'non-nmethods'"} 5828608.0
jvm_memory_pool_bytes_max{pool="Metaspace"} -1.0
jvm_memory_pool_bytes_max{pool="CodeHeap 'profiled nmethods'"} 1.22912768E8
jvm_memory_pool_bytes_max{pool="Compressed Class Space"} 1.073741824E9
jvm_memory_pool_bytes_max{pool="G1 Eden Space"} -1.0
jvm_memory_pool_bytes_max{pool="G1 Old Gen"} 1.002438656E9
```

Step:17 we can see the docker image is push to docker hub registry

Repositories | rahulf121

hub.docker.com/repositories/rahulf121

New Insights on the state of AI agents from 800+ builders and leaders. Download your copy →

Explore My Hub

Search Docker Hub

rahulf121 Your personal account

Repositories

Hardened Images

Collaborations

Settings

Default privacy

Notifications

Billing

Usage

Pulls

Storage

Upgrade plan

Repositories

All repositories within the rahulf121 namespace.

Search by repository name

All content

Create a repository

Name	Last Pushed ↑	Contains	Visibility	Scout
rahulf121/bms	34 minutes ago	IMAGE	Public	Inactive

1-1 of 1

Nalathati Rahul

